

MATH3881/5881: Statistical Machine Learning Theory

Week 1: Introduction to Machine Learning

Sarat Moka

UNSW, Sydney

Key Topics

1.1	What is Machine Learning?	1-2
1.2	Data	1-5
1.3	Supervised Learning	1-8
1.4	Linear Regression	1-12
1.5	Logistic Regression	1-17
1.6	SoftMax Regression	1-20
1.7	Support Vector Machines	1-22
1.8	Bias–Variance Decomposition and Regularization	1-24
1.9	Gradient Descent	1-26

Books:

(A) *Mathematical Engineering of Deep Learning* book by Lique, Moka, and Nazarathy (2024).

1.1 What is Machine Learning?

The world of machine learning lies at the intersection of statistics and computer science. In statistics, *data* and *randomness* play central roles, whereas in computer science, the focus is on *algorithms* and *computation*. Machine learning combines these two perspectives—it is about designing and training mathematical models on a computer to classify data, predict outcomes, estimate relationships, summarize patterns, or control complex systems.

Machine learning involves a few key activities that occur when we train, adjust, or design models and algorithms. Some of these activities are collectively referred to as *learning*, while others focus on prediction or decision-making. Any machine learning process can be described as an interaction between four main entities: *data*, *models*, *algorithms*, and the *real world*.

- By *data*, we mean either collected examples such as feature–label pairs (x, y) or data generated by models and algorithms.
- By *models*, we refer to mathematical objects stored on a computer, characterized by a set of parameters.
- By *algorithms*, we mean the procedures used to build, train, or apply these models.
- The *real world* refers to the environments that generate data, label it, and use the model outputs for decision-making.

Remark 1.1.1

In most practical settings, these four entities are connected in a loop: real-world processes generate data, algorithms build models from the data, and the resulting models are then used to make decisions that affect the real world again. This feedback loop is what allows modern AI systems to improve continuously.

Machine learning activities are often divided into two main categories: *supervised learning* and *unsupervised learning*.

- In *supervised learning*, data is available as labeled pairs (x, y) , where each feature vector x has an associated label y .
- In *unsupervised learning*, only the feature vectors x are observed, and the goal is to uncover structure or relationships within the data.

To understand this distinction, think of how toddlers learn. Early learning is mostly unsupervised—children observe and explore their surroundings without explicit instruction. Later, in school, learning becomes supervised as teachers provide labeled examples (“this is an apple”, “that is a triangle”).

Example 1.1.1

Suppose we have a dataset of houses, each with features such as the number of rooms, size in square meters, and location. Our goal is to predict the *price* of a new house given its features. Here, the input x represents the features, the output y is the price, and our task is to learn a function $f(x)$ that best predicts y .

Exercise 1.1.1

Think about how you learned to recognize faces. Was this process closer to supervised learning, unsupervised learning, or a mixture of both? Briefly explain your reasoning.

A key goal in supervised learning is to use data to train models for *prediction*. This prediction task is called *classification* when the labels y belong to a finite discrete set, and *regression* when the labels y are continuous. Trained models can then be used to make predictions for unseen, unlabeled data from the real world. Both training and prediction rely on algorithms—sometimes the trained model itself is also called an “algorithm”, especially when integrated into larger systems. Refer to Figure 1.1 for an illustration of supervised learning tasks.

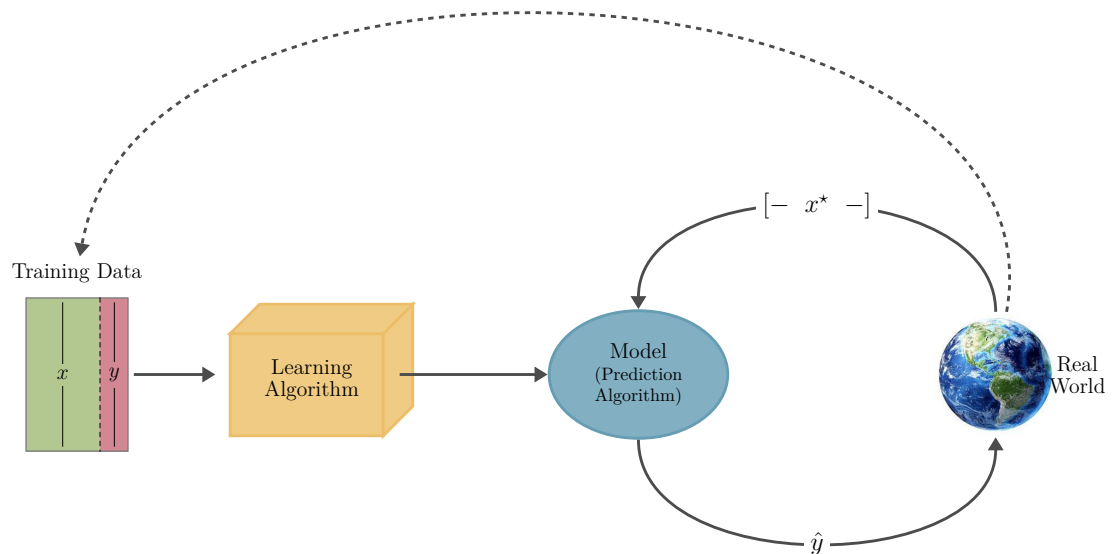


Figure 1.1: Supervised learning involves two main stages: training a model and using it for prediction.

Example 1.1.2

Classification: Predicting if a patient has a disease (yes/no).

Regression: Predicting the patient’s blood pressure based on medical measurements.

In both cases, the algorithm learns from labeled examples (x, y) .

Exercise 1.1.2

Consider a dataset with images of handwritten digits (0–9). How would you set up a supervised learning problem for this dataset? What would be your inputs x , outputs y , and possible model types?

Unsupervised learning, on the other hand, focuses on discovering structure within the data. Two common activities are *clustering*, where similar samples are grouped together, and *dimensionality reduction*, where high-dimensional data is represented in a lower-dimensional form while preserving key relationships. In these settings, the outputs are often transformed datasets rather than explicit models. Refer to Figure 1.2 for an illustration of unsupervised learning tasks.

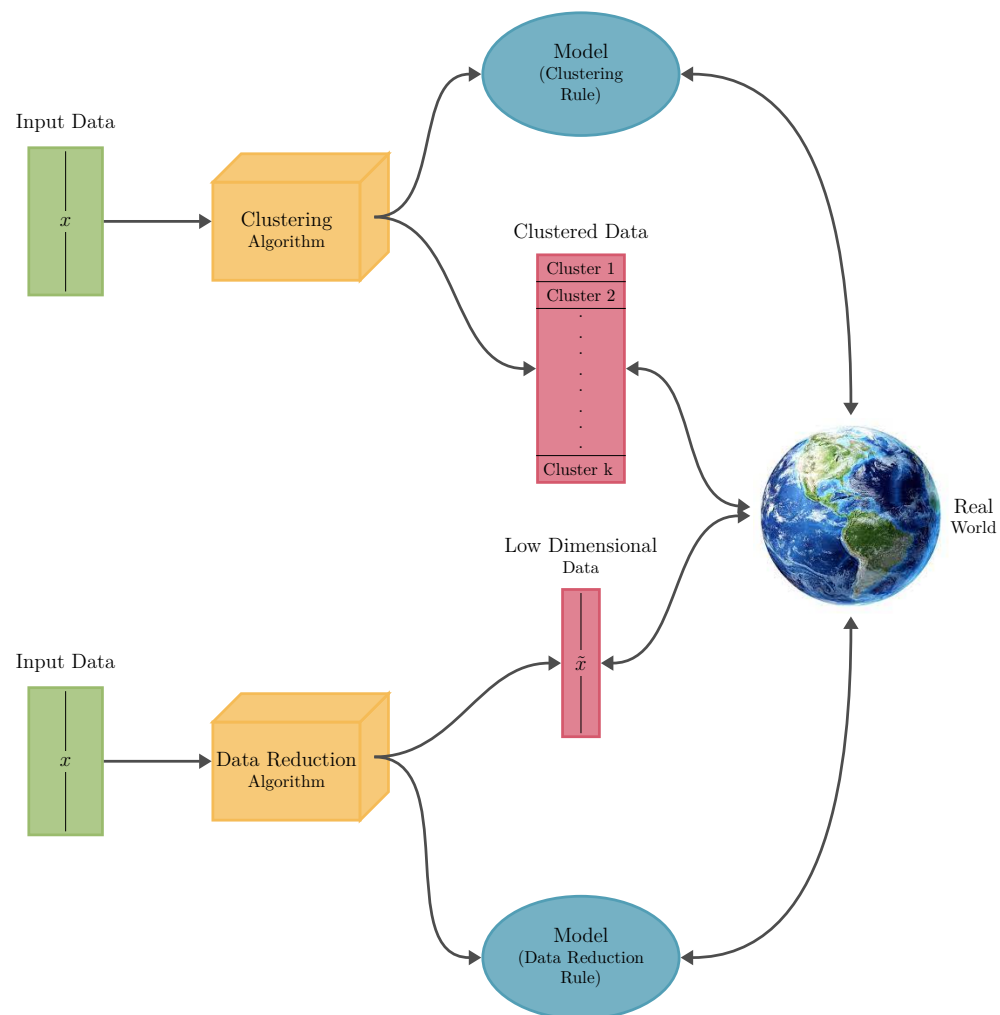


Figure 1.2: Examples of unsupervised learning: clustering (top) and dimensionality reduction (bottom).

Example 1.1.3

Supervised learning: Predicting whether an email is spam or not based on a labeled dataset of emails.

Unsupervised learning: Grouping customers into segments based on purchasing behavior without any predefined labels.

Remark 1.1.2

Clustering algorithms, such as *k*-means, group data points with similar features, while dimensionality reduction methods like *principal component analysis* (PCA) compress data into fewer dimensions while preserving its main structure. Both help us understand data without predefined labels.

Beyond supervised and unsupervised learning, there are other important paradigms:

- *Reinforcement learning* — where an agent learns by interacting with an environment over time.
- *Generative modeling* — where models learn to generate new data similar to the training data (e.g., diffusion models, generative adversarial networks (GANs)).
- *Transfer learning* — where knowledge from one domain is adapted to another.
- *Active and semi-supervised learning* — where models exploit both labeled and unlabeled data (semi-supervised learning) and may selectively query an oracle for labels of informative samples (active learning).
- *Self-supervised learning* — where models are trained on unlabeled data by creating learning targets from the data itself (e.g., masked-token prediction in BERT-style language models, next-token prediction in autoregressive language models).

Remark 1.1.3

Modern AI systems often combine multiple learning paradigms. For example, large language models use *self-supervised learning* during pretraining and *reinforcement learning* during fine-tuning (e.g., from human feedback).

1.2 Data

Data is central to every machine learning activity. It is useful to distinguish between *seen data* and *unseen data*.

- *Seen data* refers to the data available during the learning phase. This includes data used for training models, tuning parameters, and evaluating performance.

- *Unseen data* refers to new data from the real world that was not available during training. It can include data collected later or data from slightly different conditions.

The key assumption in machine learning is that the seen and unseen data come from similar underlying processes. If this assumption fails, the trained model may perform poorly when deployed.

Example 1.2.1

Suppose we train a model on weather data collected in Sydney during the past 10 years. If we later apply it to predict weather in another region or under drastically changing climate conditions, its performance may drop because the unseen data no longer resembles the training data.

To evaluate model performance realistically, we split the seen data into subsets:

- *Training set* — used to fit or learn model parameters.
- *Test set* (or *hold-out set*) — used to simulate unseen data and evaluate generalization.

Remark 1.2.1

When the test set is used repeatedly to tune model parameters, it no longer represents unseen data. To avoid this issue, practitioners often create a third subset called the *validation set*, used for hyperparameter tuning, or use *K-fold cross-validation*.

Exercise 1.2.1

You have a dataset with 10,000 labeled samples. If you use 80% for training and 20% for testing, how many samples go into each set? What could happen if you used all the data for training without keeping a test set?

In situations where data is scarce (e.g., biomedical experiments), splitting data may not be feasible. Instead, model evaluation may rely on statistical criteria such as the *Akaike Information Criterion* (*AIC*) or the *Bayesian Information Criterion* (*BIC*), which adjust for model complexity.

Preprocessing

Raw data often requires *preprocessing* before it can be used effectively for training or prediction. A common preprocessing step is *standardization* (or *normalization*), where each feature is centered and scaled.

For a feature i with values $x_i^{(1)}, \dots, x_i^{(n)}$, we compute:

$$\bar{x}_i = \frac{1}{n} \sum_{j=1}^n x_i^{(j)}, \quad s_i^2 = \frac{1}{n} \sum_{j=1}^n (x_i^{(j)} - \bar{x}_i)^2. \quad (1.1)$$

Then, each standardized value is

$$z_i^{(j)} = \frac{x_i^{(j)} - \bar{x}_i}{s_i}, \quad j = 1, \dots, n. \quad (1.2)$$

After standardization, each feature has mean 0 and variance 1. If the data is approximately normal, most standardized samples lie in the range $[-3, 3]$.

Remark 1.2.2

Standardization ensures that all features contribute comparably to the learning process. Without it, features with large numerical scales may dominate the model. It also improves numerical stability and convergence of optimization algorithms.

Example 1.2.2

Other normalization techniques: Besides standardization, other popular transformations include:

- *Min-max scaling:* $z_i^{(j)} = \frac{x_i^{(j)} - \min(x_i)}{\max(x_i) - \min(x_i)}$, which maps values to $[0, 1]$.
- *Robust scaling:* uses the median and interquartile range instead of mean and variance, reducing the influence of outliers.
- *Unit vector scaling:* rescales feature vectors to have unit length, often used in text or image feature processing.

Exercise 1.2.2

Compute the standardized version of the following feature values:

$$x = [2, 4, 6, 8, 10].$$

Verify that the standardized feature has mean 0 and variance 1.

Learning $\approx$ Optimization

Most learning algorithms involve some form of *optimization*. The goal is to find model parameters that minimize a given objective or *loss function*. For example, in regression tasks, one might minimize the *mean squared error*:

$$\frac{1}{n} \sum_{i=1}^n (y_i - f(x_i))^2.$$

In classification, the loss could be the *cross-entropy* between predicted and true labels.

Remark 1.2.3

Learning as optimization: Have you studied linear regression in the past? If yes, recall that training a linear regression model means finding coefficients that minimize the average squared difference between predicted and observed values. This is an explicit optimization problem over the model parameters.

Even algorithms that do not specify an optimization problem directly, such as decision trees, implicitly perform optimization through greedy or heuristic procedures.

Remark 1.2.4

Optimization connects mathematics, statistics, and computation in machine learning. In later weeks, we will study gradient descent, adaptive methods, and second-order techniques that form the backbone of modern deep learning.

Exercise 1.2.3

You must have studied some loss functions in your previous courses. Give one example each of:

1. A loss function used in regression.
2. A loss function used in classification.

Explain what minimizing each loss function achieves.

1.3 Supervised Learning

Supervised learning is the task of learning a mapping from inputs to outputs using a labeled dataset. Formally, we are given *training data*

$$\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^n, \quad (1.3)$$

where each $x^{(i)} \in \mathbb{R}^p$ is a *feature vector* and $y^{(i)} \in \mathcal{Y}$ is its associated *label* or *target*. The output space $\mathcal{Y}$ depends on the task:

- $\mathcal{Y} = \mathbb{R}$ for *regression* (predicting a continuous quantity),
- $\mathcal{Y} = \{0, 1\}$ for *binary classification*,
- $\mathcal{Y} = \{1, \dots, K\}$ for *multi-class classification* with K classes.

The goal is to use $\mathcal{D}$ to find a function f that predicts y accurately for new, unseen inputs x .

The Model and Its Parameters

We restrict our search to a *parameterized family* of functions f_θ , indexed by a parameter vector $\theta \in \Theta \subseteq \mathbb{R}^d$. Choosing a family is what we mean by choosing a *model*; finding a specific θ within that family is what we mean by *learning* or *fitting* the model.

Example 1.3.1

A linear model takes the form $f_\theta(x) = \theta^\top \tilde{x}$, where $\tilde{x} = (1, x_1, \dots, x_p)^\top \in \mathbb{R}^{p+1}$ and $\theta \in \mathbb{R}^{p+1}$. Here the parameter space is $\Theta = \mathbb{R}^{p+1}$, and learning means finding the best θ in this family.

Remark 1.3.1

The choice of model family encodes our assumptions about the relationship between x and y . A linear model assumes this relationship can be captured by a weighted sum of features. More expressive families — such as deep neural networks — make fewer structural assumptions but require more data and computation to train reliably.

From Raw Scores to Predictions

The raw output $f_\theta(x)$ is a real number (or a vector for multi-class tasks), but it is not always directly interpretable as a prediction. Depending on the task, we apply an *output transformation* to produce the final prediction $\hat{y}$:

- *Regression*: No transformation is needed. We set $\hat{y} = f_\theta(x) \in \mathbb{R}$ directly.
- *Binary classification*: We apply the *sigmoid* function

$$\hat{y} = \sigma(f_\theta(x)) = \frac{1}{1 + e^{-f_\theta(x)}} \in (0, 1), \quad (1.4)$$

which squashes any real number into $(0, 1)$, interpretable as $\hat{\mathbb{P}}(y = 1 \mid x)$.

- *Multi-class classification (K classes)*: The model produces a score vector $s = f_\theta(x) \in \mathbb{R}^K$, one score per class. We apply the *softmax* function

$$\hat{y}_k = \text{softmax}(s)_k = \frac{e^{s_k}}{\sum_{j=1}^K e^{s_j}}, \quad k = 1, \dots, K, \quad (1.5)$$

producing a vector $\hat{y} \in (0, 1)^K$ with $\sum_{k=1}^K \hat{y}_k = 1$, interpretable as a probability distribution over the K classes.

Remark 1.3.2

The sigmoid (1.4) is a special case of the softmax (1.5) for $K = 2$. Both transformations are smooth and strictly monotone in the scores, making them compatible with gradient-based optimization.

Loss Functions

A *loss function* $\ell(y, \hat{y})$ measures how far the model's prediction $\hat{y}$ is from the true label y on a single example. A good loss function should:

- reflect the cost of prediction errors for the task at hand,
- be differentiable (or at least sub-differentiable) to allow gradient-based optimization,
- be convex in θ where possible, ensuring that any local minimum is a global one.

Table 1.1 summarizes the canonical loss for each of the three tasks we study.

Task	Loss name	Formula $\ell(y, \hat{y})$	Model
Regression	Squared (L_2)	$(y - \hat{y})^2$	Linear regression
Binary classification	Binary cross-entropy	$-y \log \hat{y} - (1 - y) \log(1 - \hat{y})$	Logistic regression
Multi-class classification	Categorical cross-entropy	$-\sum_{k=1}^K y_k \log \hat{y}_k$	SoftMax regression

Table 1.1: Canonical loss functions for the three supervised learning tasks. Here $\hat{y}$ is the transformed model output (identity, sigmoid, or softmax respectively). In the multi-class case, $y \in \{0, 1\}^K$ is the *one-hot* encoding of the true class.

Remark 1.3.3

Connection to maximum likelihood estimation. All three losses in Table 1.1 are instances of the *negative log-likelihood* under a standard distributional model: the squared loss corresponds to a Gaussian noise model on y , binary cross-entropy to a Bernoulli model, and categorical cross-entropy to a categorical model. This probabilistic connection is not a coincidence — it provides a principled way to derive loss functions from assumptions about how labels are generated.

Exercise 1.3.1

Look at the binary cross-entropy loss in Table 1.1.

1. What is the loss when $y = 1$ and $\hat{y} \rightarrow 1$? When $y = 1$ and $\hat{y} \rightarrow 0$?
2. Why is the loss undefined at $\hat{y} = 0$ or $\hat{y} = 1$, and why does this not cause problems in practice when $\hat{y} = \sigma(f_\theta(x))$?

Empirical Risk Minimization

Given a loss function ℓ , the *empirical risk* (or *training loss*) is the average loss over the training data:

$$C(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(y^{(i)}, f_{\theta}(x^{(i)})). \quad (1.6)$$

Learning the model means finding parameters $\hat{\theta}$ that minimize the empirical risk:

$$\hat{\theta} = \arg \min_{\theta \in \Theta} C(\theta). \quad (1.7)$$

This principle is known as *Empirical Risk Minimization* (ERM) and is the organizing idea behind most supervised learning algorithms.

Remark 1.3.4

Empirical risk vs. population risk. The true goal of learning is to minimize the *population risk*

$$R(\theta) = \mathbb{E}_{(x,y)} [\ell(y, f_{\theta}(x))],$$

the expected loss over all possible data drawn from the underlying distribution. Since this distribution is unknown, we minimize $C(\theta)$ — the best proxy available from finite data. The gap between $C(\theta)$ and $R(\theta)$ is the central object of study in statistical learning theory.

Exercise 1.3.2

1. For each loss in Table 1.1, write out the empirical risk $C(\theta)$ explicitly as a sum over the n training examples.
2. When f_{θ} is a linear model, which of the three losses yields a convex objective in θ ?

Remark 1.3.5

Road map. The rest of this week instantiates the ERM framework for three fundamental models:

- *Linear regression:* $f_{\theta}(x) = \theta^{\top} \tilde{x}$, squared loss. Admits a closed-form solution via the normal equations.
- *Logistic regression:* $\hat{y} = \sigma(\theta^{\top} \tilde{x})$, binary cross-entropy. No closed form; solved via gradient descent.
- *SoftMax regression:* $\hat{y} = \text{softmax}(W\tilde{x})$, categorical cross-entropy. Extends logistic regression to K classes; solved via gradient descent.

These three models are the building blocks of modern neural networks: a deep network is essentially a composition of such layers, with the final layer being one of these three depending on the task.

1.4 Linear Regression

Linear regression is the instantiation of the ERM framework (Section 1.3) for $\mathcal{Y} = \mathbb{R}$: we use a linear model f_θ and the squared loss. The result is one of the most fundamental and well-understood algorithms in machine learning, serving as the template for logistic regression, softmax regression, and ultimately deep neural networks.

To build intuition, consider the *univariate* ($p = 1$) problem of predicting the median home value (`medv`) of a Boston suburb from the average number of rooms per dwelling (`rm`). The *simple linear regression* model posits

$$y = \theta_0 + \theta_1 x + \epsilon, \quad (1.8)$$

where $\theta_0 \in \mathbb{R}$ is the *intercept*, $\theta_1 \in \mathbb{R}$ is the *slope*, and ϵ captures variation not explained by x . Learning means finding $\hat{\theta} = (\hat{\theta}_0, \hat{\theta}_1)$ that best fits the data; see Figure 1.5(a).

Remark 1.4.1

In statistical inference, distributional assumptions on ϵ (e.g., $\epsilon \sim \mathcal{N}(0, \sigma^2)$) enable confidence intervals and hypothesis tests. In ML practice, one typically forgoes such assumptions and focuses purely on predictive performance.

The Linear Model

For the general case $x = (x_1, \dots, x_p)^\top \in \mathbb{R}^p$ and $y \in \mathbb{R}$, define the *augmented input* $\tilde{x} = (1, x_1, \dots, x_p)^\top \in \mathbb{R}^{p+1}$ and the *parameter vector* $\theta = (\theta_0, \theta_1, \dots, \theta_p)^\top \in \mathbb{R}^{p+1}$. Setting $d = p + 1$, the linear model is

$$f_\theta(x) = \theta^\top \tilde{x} = \theta_0 + \theta_1 x_1 + \dots + \theta_p x_p, \quad (1.9)$$

where θ_0 is the *intercept* (or *bias*) and $\theta_1, \dots, \theta_p$ are the *weights*. Given an estimated $\hat{\theta}$, the prediction for a new input x^* is $\hat{y}(x^*) = \hat{\theta}^\top \tilde{x}^*$.

Remark 1.4.2

The model is called *linear* because $f_\theta(x)$ is a linear function of the parameters θ . As we will see in the Feature Engineering subsection, the inputs $\tilde{x}$ can themselves be nonlinear transformations of the original data, so the model is far more flexible than the name suggests.

The Design Matrix and Least Squares

Given training data $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^n$, stack the observations into the *design matrix* $X \in \mathbb{R}^{n \times d}$ and *response vector* $y \in \mathbb{R}^n$:

$$X = \begin{bmatrix} 1 & x_1^{(1)} & \dots & x_p^{(1)} \\ \vdots & \vdots & & \vdots \\ 1 & x_1^{(n)} & \dots & x_p^{(n)} \end{bmatrix}, \quad y = \begin{bmatrix} y^{(1)} \\ \vdots \\ y^{(n)} \end{bmatrix}. \quad (1.10)$$

The vector of model predictions on all training points is $\hat{y} = X\theta \in \mathbb{R}^n$.

Instantiating the empirical risk (1.6) with the squared loss gives the *least-squares* objective:

$$C(\theta) = \frac{1}{n} \sum_{i=1}^n (y^{(i)} - \theta^\top \tilde{x}^{(i)})^2 = \frac{1}{n} \|y - X\theta\|^2. \quad (1.11)$$

Learning the model means solving

$$\hat{\theta} = \arg \min_{\theta \in \mathbb{R}^d} \|y - X\theta\|^2. \quad (1.12)$$

The Closed-Form Solution

The objective (1.11) is a convex quadratic function of θ , so its minimum is characterized by setting the gradient to zero:

$$\nabla_{\theta} C(\theta) = \frac{2}{n} (X^\top X \theta - X^\top y) = 0. \quad (1.13)$$

This yields the *normal equations*:

$$X^\top X \theta = X^\top y. \quad (1.14)$$

When X has full column rank (equivalently, $X^\top X$ is invertible), the unique minimizer is

$$\hat{\theta} = (X^\top X)^{-1} X^\top y = X^\dagger y, \quad (1.15)$$

where $X^\dagger = (X^\top X)^{-1} X^\top$ is the *Moore–Penrose pseudoinverse* of X .

When $X^\top X$ is singular — for example when $n < d$ or when features are linearly dependent — infinitely many solutions exist. Among these, the minimum-norm solution is obtained via the *singular value decomposition* (SVD). Write $X = U\Delta V^\top$, where $U \in \mathbb{R}^{n \times n}$ and $V \in \mathbb{R}^{d \times d}$ are orthogonal, and $\Delta \in \mathbb{R}^{n \times d}$ is diagonal with non-negative entries $\sigma_1 \geq \dots \geq \sigma_r > 0 = \sigma_{r+1} = \dots$. The pseudoinverse is

$$X^\dagger = V \Delta^+ U^\top, \quad (1.16)$$

where Δ^+ replaces each non-zero singular value σ_k with $1/\sigma_k$, and leaves zero entries as zero. The minimum-norm least-squares solution is then $\hat{\theta} = X^\dagger y$.

Exercise 1.4.1

1. Verify that $\nabla_{\theta} \|y - X\theta\|^2 = -2X^\top y + 2X^\top X\theta$ and derive the normal equations (1.14).
2. Under what condition on X does the solution (1.15) exist uniquely?
3. Show that $\hat{\theta} = X^\dagger y$ always satisfies the normal equations, and that it is the solution with the smallest Euclidean norm when there are infinitely many solutions.

A geometric view is given in Figure 1.3: each squared residual $(y^{(i)} - \hat{y}^{(i)})^2$ is the area of a square with side equal to the residual magnitude, and the least-squares estimator minimizes the total area.

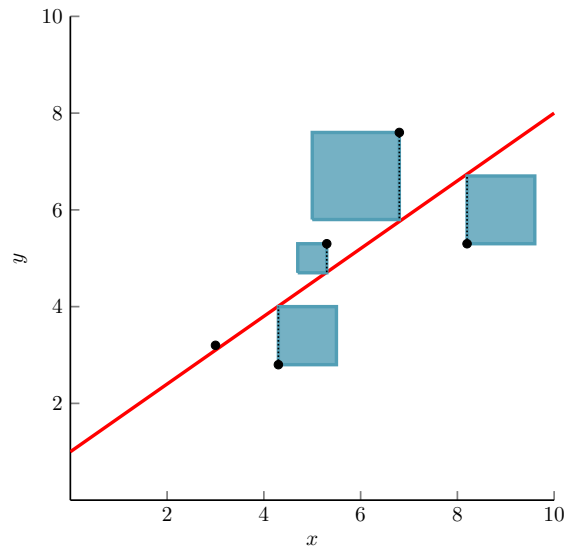


Figure 1.3: Geometric interpretation of the squared loss for $p = 1$: the least-squares estimator minimizes the total area of the squares.

Other Regression Loss Functions

The squared loss is analytically convenient and yields the closed-form solution (1.15), but it is sensitive to outliers: a single large residual contributes quadratically to $C(\theta)$ and can dominate the fit. Two robust alternatives are widely used.

Absolute error (L_1) loss.

$$\ell(y^{(i)}, \hat{y}^{(i)}) = |y^{(i)} - \hat{y}^{(i)}|. \quad (1.17)$$

Residuals are penalized linearly, so the influence of large outliers grows much more slowly than under L_2 . No closed form exists; the problem is solved via convex optimization.

Huber loss. For a threshold $\delta > 0$,

$$\ell(y^{(i)}, \hat{y}^{(i)}) = \begin{cases} \frac{1}{2}(y^{(i)} - \hat{y}^{(i)})^2, & \text{if } |y^{(i)} - \hat{y}^{(i)}| < \delta, \\ \delta |y^{(i)} - \hat{y}^{(i)}| - \frac{1}{2}\delta^2, & \text{otherwise.} \end{cases} \quad (1.18)$$

The Huber loss is quadratic (smooth, like L_2) near zero and linear (robust, like L_1) in the tails, with δ controlling the transition. It is differentiable everywhere, making it suitable for gradient-based optimization.

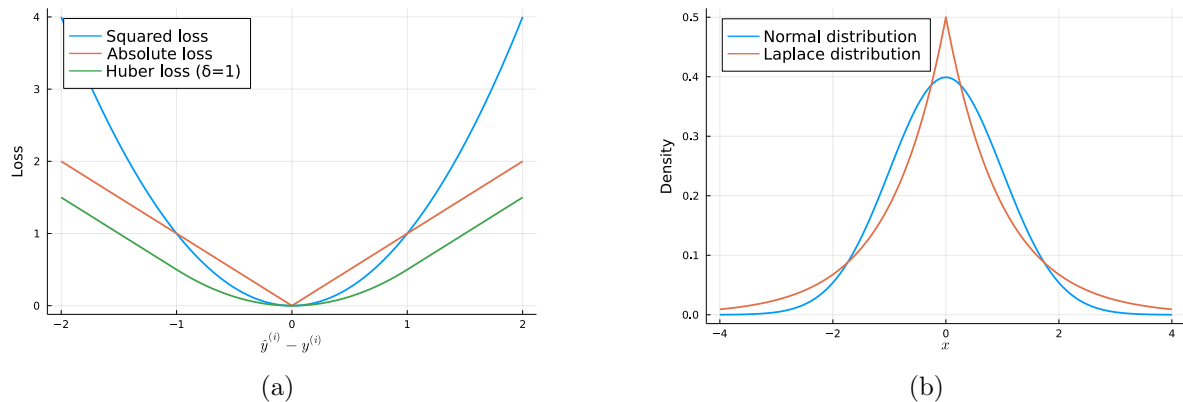


Figure 1.4: (a) Squared (L_2), absolute (L_1), and Huber losses as a function of the residual $r = y - \hat{y}$. (b) The corresponding noise distributions: Gaussian (squared loss) and Laplace (L_1 loss).

Exercise 1.4.2

1. Explain why L_1 loss is more robust to outliers than L_2 loss. What is the gradient of each loss with respect to the residual $r = y^{(i)} - \hat{y}^{(i)}$ for large $|r|$?
2. Show that the Huber loss (1.18) is differentiable everywhere by verifying continuity of the loss and its derivative at $|r| = \delta$.

Feature Engineering

The model (1.9) is *linear in the parameters* θ , but the features $\tilde{x}$ can be any fixed transformations of the raw input. This flexibility, known as *feature engineering*, allows linear models to capture nonlinear relationships in the data.

For instance, suppose a quadratic relationship is more appropriate for a univariate input:

$$y = \theta_0 + \theta_1 x + \theta_2 x^2 + \epsilon. \quad (1.19)$$

This is still linear in $\theta = (\theta_0, \theta_1, \theta_2)^\top$, with engineered features $(1, x, x^2)$. The design matrix (1.10), normal equations (1.14), and closed-form solution (1.15) apply without modification. More generally, a *degree- q polynomial* model uses features $(1, x, x^2, \dots, x^q)$, yielding $d = q + 1$ parameters. For multivariate inputs, one may add interaction terms (e.g., $x_i x_j$), trigonometric features, or any other fixed transformations.

Example 1.4.1

Figure 1.5 shows two regressions on the Boston housing data. (a) `medv` vs. `rm`: the relationship is approximately linear, and an affine fit captures it well. (b) `medv` vs. `lstat` (percentage of lower-status population): the trend is nonlinear. A linear fit (red) underfits; including a quadratic term x^2 (blue) gives a substantially better fit, illustrating the benefit of feature engineering.

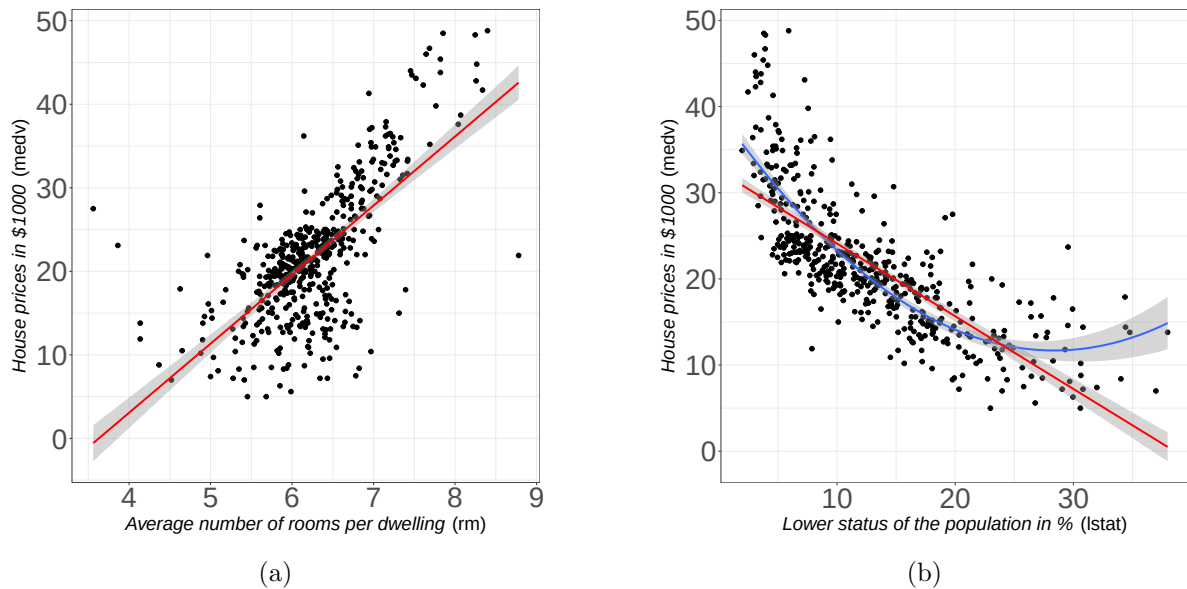


Figure 1.5: Boston housing data. (a) `medv` vs. `rm`: an affine fit captures the linear trend. (b) `medv` vs. `lstat`: a linear fit (red) underfits; a quadratic fit (blue) is more appropriate.

Remark 1.4.3

Adding more features increases the expressiveness of the model and the number of parameters d , raising the risk of *overfitting*: fitting the training data well but generalizing poorly to unseen data. The trade-off between model complexity and generalization is a central theme in machine learning, addressed through regularization and cross-validation.

Exercise 1.4.3

1. Write the design matrix X for the degree-2 polynomial model (1.19) applied to n training points $\{(x^{(j)}, y^{(j)})\}_{j=1}^n$.
2. Derive the normal equations (1.14) for this model and write the closed-form solution for $\hat{\theta} = (\hat{\theta}_0, \hat{\theta}_1, \hat{\theta}_2)^\top$.

Categorical Input Features

Not all features are numerical. *Categorical features* take values in a finite unordered set (e.g., colour, city, job type) and must be encoded as numbers before they can enter the design matrix.

Binary and ordinal features. A binary feature (e.g., **yes/no**) is naturally mapped to $\{0, 1\}$. An ordinal feature with a natural ordering (e.g., **low** < **medium** < **high**) can be mapped to consecutive integers, though this implicitly assumes equal spacing between levels.

One-hot encoding. For a nominal feature $z \in \{1, \dots, L\}$ with no natural ordering, replace z with the *one-hot vector* $\tilde{z} = e_z \in \{0, 1\}^L$, where e_z is the z -th standard basis vector. If the dataset has p_{num} numerical features and categorical features with $L_1, \dots, L_k$ levels respectively, the total number of encoded features is

$$\tilde{p} = p_{\text{num}} + \sum_{i=1}^k L_i.$$

To avoid perfect collinearity with the intercept column, one level per feature is typically designated as the *reference category* and its indicator is dropped, leaving $L_i - 1$ dummy variables per feature.

Exercise 1.4.4

1. A dataset has one numerical feature and one categorical feature with $L = 4$ levels. Write the design matrix $X \in \mathbb{R}^{n \times d}$ using $L - 1$ dummy variables (plus intercept), and state the value of d .
2. Explain why including all L dummy variables *and* an intercept column makes X rank-deficient, and why this causes the normal equations (1.14) to have no unique solution.

1.5 Logistic Regression

Logistic regression is the instantiation of the ERM framework for binary classification: $\mathcal{Y} = \{0, 1\}$. We use the sigmoid output transformation and binary cross-entropy loss introduced in Section 1.3.

The Model

Given an input $x \in \mathbb{R}^p$, logistic regression models the probability of the positive class as

$$\hat{y} = \sigma(\theta^\top \tilde{x}) = \frac{1}{1 + e^{-\theta^\top \tilde{x}}} \in (0, 1), \quad (1.20)$$

where $\tilde{x} = (1, x_1, \dots, x_p)^\top$ is the augmented input and $\theta \in \mathbb{R}^{p+1}$ are the model parameters. The output $\hat{y}$ is interpreted as $\hat{\mathbb{P}}(y = 1 | x)$, the estimated probability that the input belongs to the positive class.

The term $\theta^\top \tilde{x}$ is called the *log-odds* or *logit*, since

$$\theta^\top \tilde{x} = \log \frac{\hat{y}}{1 - \hat{y}}.$$

Each weight θ_j encodes the change in log-odds per unit increase in feature x_j , holding all other features fixed.

Remark 1.5.1

Despite its name, logistic regression is a *classification* model. The name comes from the use of the logistic (sigmoid) function to map a linear score to a probability.

The Binary Cross-Entropy Loss and ERM

Substituting the binary cross-entropy loss from Table 1.1 and the model (1.20) into the empirical risk (1.6) gives

$$C(\theta) = -\frac{1}{n} \sum_{i=1}^n \left[y^{(i)} \log \sigma(\theta^\top \tilde{x}^{(i)}) + (1 - y^{(i)}) \log (1 - \sigma(\theta^\top \tilde{x}^{(i)})) \right]. \quad (1.21)$$

Learning the model means solving

$$\hat{\theta} = \arg \min_{\theta \in \mathbb{R}^{p+1}} C(\theta). \quad (1.22)$$

Remark 1.5.2

The objective $C(\theta)$ in (1.21) is *convex* in θ , so any local minimum is a global minimum. Unlike linear regression with squared loss, however, there is no closed-form solution: the sigmoid introduces nonlinearity that prevents an algebraic solution. The minimization (1.22) can therefore be solved iteratively via gradient descent (Section 1.9).

Exercise 1.5.1

Using the identity $\sigma'(z) = \sigma(z)(1 - \sigma(z))$:

1. Show that the binary cross-entropy loss for a single example can be written as

$$\ell(y^{(i)}, \hat{y}^{(i)}) = \log \left(1 + e^{\theta^\top \tilde{x}^{(i)}} \right) - y^{(i)} \theta^\top \tilde{x}^{(i)}.$$

2. Verify that this is a convex function of θ by computing the Hessian with respect to θ and showing it is positive semi-definite.

Making Predictions

Once $\hat{\theta}$ is learned, the model outputs a predicted probability $\hat{y} = \sigma(\hat{\theta}^\top \tilde{x}) \in (0, 1)$ for each input x . To produce a class label, we apply a *decision threshold* $\tau \in (0, 1)$:

$$\hat{c}(x) = \begin{cases} 1 \text{ (positive)}, & \text{if } \hat{y} > \tau, \\ 0 \text{ (negative)}, & \text{if } \hat{y} \leq \tau. \end{cases} \quad (1.23)$$

The default $\tau = 0.5$ predicts the more probable class. Adjusting τ trades off between false positives and false negatives, and is chosen based on the relative costs of each error type in the application.

Evaluation Metrics

Given predictions $\{\hat{c}(x^{(i)})\}$ on a test set $\{(x^{(i)}, y^{(i)})\}_{i=1}^{n_{\text{test}}}$, define the four counts:

$$\text{TP} = \#\{\hat{c} = 1, y = 1\}, \quad \text{TN} = \#\{\hat{c} = 0, y = 0\}, \quad \text{FP} = \#\{\hat{c} = 1, y = 0\}, \quad \text{FN} = \#\{\hat{c} = 0, y = 1\}.$$

The standard performance metrics are:

$$\begin{aligned} \text{Accuracy} &= \frac{\text{TP} + \text{TN}}{n_{\text{test}}}, & \text{Precision} &= \frac{\text{TP}}{\text{TP} + \text{FP}}, \\ \text{Recall} &= \frac{\text{TP}}{\text{TP} + \text{FN}}, & \text{Specificity} &= \frac{\text{TN}}{\text{TN} + \text{FP}}. \end{aligned}$$

The *receiver operating characteristic (ROC) curve* plots $(1 - \text{Specificity}(\tau), \text{Recall}(\tau))$ as τ varies over $(0, 1)$, tracing the full trade-off between true positive and false positive rates. The *area under the ROC curve (AUC)* summarizes overall discriminative performance: a random classifier achieves $\text{AUC} \approx 0.5$ and a perfect classifier achieves $\text{AUC} = 1$.

Remark 1.5.3

Imbalanced classes. When one class is far more frequent than the other, accuracy can be misleadingly high: a classifier that always predicts the majority class achieves high accuracy while having zero recall. In such cases, prefer Precision, Recall, Specificity, or AUC as evaluation criteria, and calibrate τ accordingly.

A common scalar summary balancing Precision and Recall is the F_1 score:

$$F_1 = \frac{2}{\frac{1}{\text{Precision}} + \frac{1}{\text{Recall}}} = 2 \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \quad (1.24)$$

The more general F_β score weights Recall by a factor $\beta > 0$ relative to Precision:

$$F_\beta = \frac{(1 + \beta^2) \cdot \text{Precision} \cdot \text{Recall}}{\beta^2 \cdot \text{Precision} + \text{Recall}}. \quad (1.25)$$

When $\beta > 1$, Recall is weighted more heavily; when $\beta < 1$, Precision is weighted more heavily; $\beta = 1$ recovers F_1 .

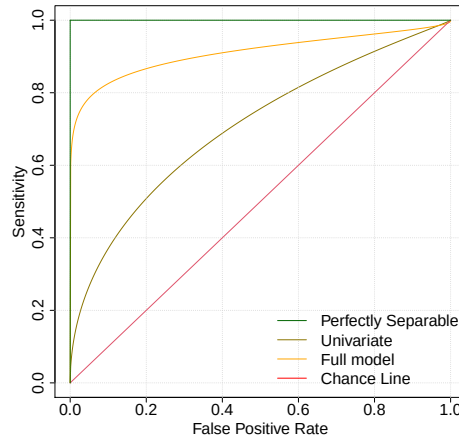


Figure 1.6: ROC curves on the Wisconsin breast cancer data for a univariate logistic model ($p = 1$) and the full model ($p = 30$). The diagonal represents a random classifier; a curve closer to the top-left corner indicates better discrimination.

Example 1.5.1

Threshold tuning. For a fixed $\hat{\theta}$, decreasing τ increases Recall (fewer false negatives) at the cost of lower Precision (more false positives). In medical screening, missing a disease is typically more costly than a false alarm, suggesting a low τ ; in spam filtering, incorrectly blocking a legitimate email may be more costly, suggesting a higher τ . The F_β score with appropriate β formalizes this trade-off.

Exercise 1.5.2

1. Given $(TP, FP, TN, FN) = (36, 8, 61, 8)$, compute Accuracy, Precision, Recall, Specificity, and F_1 .
2. Suppose τ is decreased from 0.5 to 0.3. Qualitatively, how would each metric change and why?
3. A test set has 950 negatives and 50 positives. A classifier always predicts **negative**. What is its Accuracy? What are its Recall and Precision? Why is Accuracy misleading here?

1.6 SoftMax Regression

SoftMax regression is the instantiation of the ERM framework for multi-class classification: $\mathcal{Y} = \{1, \dots, K\}$. It extends logistic regression to $K > 2$ classes using the softmax transformation and categorical cross-entropy loss.

The Model

For each class $k \in \{1, \dots, K\}$, we maintain a separate parameter vector $\theta_k \in \mathbb{R}^{p+1}$, collected into a *weight matrix* $W = [\theta_1 \mid \dots \mid \theta_K]^\top \in \mathbb{R}^{K \times (p+1)}$. Given input x , the model computes a *score vector* $s = W\tilde{x} \in \mathbb{R}^K$, where $s_k = \theta_k^\top \tilde{x}$ is the score for class k . The scores are converted to class probabilities via the softmax transformation (1.5):

$$\hat{y}_k = \frac{e^{s_k}}{\sum_{j=1}^K e^{s_j}}, \quad k = 1, \dots, K, \quad (1.26)$$

so that $\hat{y} \in (0, 1)^K$ with $\sum_{k=1}^K \hat{y}_k = 1$. Labels are represented as *one-hot vectors*: if the true class is c , then $y = e_c \in \{0, 1\}^K$.

Remark 1.6.1

Connection to logistic regression. For $K = 2$, one of the two weight vectors is redundant (only the difference $\theta_1 - \theta_2$ matters), and softmax regression reduces exactly to logistic regression with a single parameter vector $\theta \in \mathbb{R}^{p+1}$.

The Categorical Cross-Entropy Loss and ERM

With one-hot labels $y^{(i)} \in \{0, 1\}^K$, the categorical cross-entropy loss for a single example simplifies to

$$\ell(y^{(i)}, \hat{y}^{(i)}) = - \sum_{k=1}^K y_k^{(i)} \log \hat{y}_k^{(i)} = - \log \hat{y}_{c^{(i)}}^{(i)}, \quad (1.27)$$

where $c^{(i)}$ is the true class of the i -th example. The empirical risk is

$$C(W) = -\frac{1}{n} \sum_{i=1}^n \sum_{k=1}^K y_k^{(i)} \log \hat{y}_k^{(i)}, \quad (1.28)$$

and learning amounts to solving $\hat{W} = \arg \min_W C(W)$.

Remark 1.6.2

The objective $C(W)$ is convex in W and has no closed-form solution. As with logistic regression, it is minimized iteratively via gradient descent (Section 1.9).

Making Predictions

Given $\hat{W}$, the predicted class for a new input x is

$$\hat{c}(x) = \arg \max_{k \in \{1, \dots, K\}} \hat{y}_k = \arg \max_{k \in \{1, \dots, K\}} s_k, \quad (1.29)$$

where the second equality holds because softmax is monotone: it preserves the ordering of the scores. Hence prediction requires only the score vector $W\tilde{x}$, not the softmax probabilities themselves.

Remark 1.6.3

Alternative multi-class strategies. Two classical alternatives to softmax regression reduce multi-class problems to binary ones. *One-vs-rest (OvR)* trains K binary classifiers, one for each class against all others, and predicts $\hat{c}(x) = \arg \max_k f_{\theta_k}(x)$. *One-vs-one (OvO)* trains $K(K-1)/2$ binary classifiers for each pair of classes and decides by majority vote. SoftMax regression is generally preferred as it produces calibrated probabilities and is trained jointly across all classes.

Exercise 1.6.1

1. For $K = 3$ classes and $p = 2$ features, state the dimensions of W , $\tilde{x}$, s , and $\hat{y}$.
2. Show that for $K = 2$, the softmax model (1.26) reduces to the logistic regression model (1.20).
3. Why does predicting $\hat{c}(x) = \arg \max_k s_k$ give the same result as $\arg \max_k \hat{y}_k$?

1.7 Support Vector Machines

Support vector machines (SVMs) are a family of linear classifiers for binary classification ($y \in \{-1, +1\}$) based on the principle of *margin maximization*. They provide a complementary perspective to logistic regression and connect naturally to regularization.

Hard-Margin SVM

Suppose the training data are *linearly separable*: there exists a hyperplane $\{x : w^\top x + b = 0\}$ that correctly separates the two classes. Among all such hyperplanes, the SVM selects the one with the largest *geometric margin* — the minimum distance from any training point to the decision boundary.

After normalizing so that the closest points satisfy $y^{(i)}(w^\top x^{(i)} + b) = 1$, the margin equals $2/\|w\|$ and the *hard-margin SVM* solves

$$\min_{w \in \mathbb{R}^p, b \in \mathbb{R}} \frac{1}{2} \|w\|^2 \quad \text{subject to} \quad y^{(i)}(w^\top x^{(i)} + b) \geq 1, \quad i = 1, \dots, n. \quad (1.30)$$

The training points that lie exactly on the margin boundaries (i.e., for which equality holds) are called *support vectors*; all others do not affect the solution.

Remark 1.7.1

Problem (1.30) is a convex quadratic program and can be solved efficiently via its Lagrangian dual, which reveals that the solution $\hat{w}$ is a linear combination of only the support vectors.

Soft-Margin SVM

When the data are not linearly separable, the hard-margin constraints have no feasible solution. The *soft-margin SVM* relaxes them by introducing *slack variables* $\xi_i \geq 0$, allowing some points to violate the margin:

$$\min_{w, b, \xi} \frac{1}{2} \|w\|^2 + c \sum_{i=1}^n \xi_i \quad \text{subject to} \quad y^{(i)}(w^\top x^{(i)} + b) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad (1.31)$$

where $c > 0$ is a hyperparameter trading off margin width against training errors. Large c penalizes violations heavily (narrow margin, low training error); small c allows more violations (wide margin, higher training error).

SVM as ERM with Hinge Loss

The soft-margin SVM (1.31) has an equivalent unconstrained formulation that connects it directly to the ERM framework. Defining the *hinge loss*

$$\ell(y^{(i)}, \hat{y}^{(i)}) = \max(0, 1 - y^{(i)}\hat{y}^{(i)}), \quad \hat{y}^{(i)} = w^\top x^{(i)} + b, \quad (1.32)$$

and setting $\lambda = 1/(2nc)$, the soft-margin SVM is equivalent to

$$\min_{w, b} \frac{1}{n} \sum_{i=1}^n \max(0, 1 - y^{(i)}(w^\top x^{(i)} + b)) + \lambda \|w\|^2. \quad (1.33)$$

This is precisely ERM (1.35) with hinge loss and L_2 regularization on w .

Remark 1.7.2

SVM vs. logistic regression. Both are linear classifiers that minimize a convex surrogate loss plus (optionally) L_2 regularization. The key difference is the loss: logistic regression uses binary cross-entropy (smooth, assigns non-zero loss to all points), while SVM uses hinge loss (piecewise linear, assigns zero loss to correctly classified points far from the margin). SVM is therefore *sparse* in the sense that only support vectors influence $\hat{w}$.

Exercise 1.7.1

Explain the role of c in (1.31): what does the solution look like as $c \rightarrow \infty$ and as $c \rightarrow 0$?

1.8 Bias–Variance Decomposition and Regularization

The Bias–Variance Trade-off

ERM minimizes the empirical risk $C(\theta)$ on training data, but the ultimate goal is low *population risk* $R(\theta)$. The gap between the two — and the difficulty of closing it — is explained by the *bias–variance decomposition*.

Suppose labels are generated as $y = f^*(x) + \epsilon$, where f^* is the true function and ϵ is zero-mean noise with variance σ^2 . Let $\hat{f}_{\mathcal{D}}(x)$ denote the prediction at x of a model trained on a dataset $\mathcal{D}$ of size n . The expected squared prediction error at a fixed test point x , averaged over random training sets and the test-point noise ϵ , decomposes as:

$$\mathbb{E}_{\mathcal{D}, \epsilon}[(y - \hat{f}_{\mathcal{D}}(x))^2] = \underbrace{\left(f^*(x) - \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(x)]\right)^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}_{\mathcal{D}}\left[(\hat{f}_{\mathcal{D}}(x) - \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(x)])^2\right]}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{Noise}}. \quad (1.34)$$

- *Bias* measures how far the average prediction is from the truth: a systematic error due to model assumptions. A model that is too simple (e.g., fitting a line to a nonlinear relationship) has high bias.
- *Variance* measures how much the prediction fluctuates across different training sets: sensitivity to the particular data seen. A model that is too complex (e.g., a high-degree polynomial with few data points) has high variance.
- *Noise* σ^2 is irreducible: it cannot be eliminated by any model.

Remark 1.8.1

The bias–variance trade-off. Increasing model complexity (more parameters, richer feature maps) typically decreases bias but increases variance. Decreasing complexity does the opposite. The optimal model balances the two: simple enough to generalize, expressive enough to capture the true structure. This trade-off underlies the phenomena of *underfitting* (high bias) and *overfitting* (high variance).

Example 1.8.1

For polynomial regression of degree q on a fixed dataset: a degree-1 model may underfit (high bias, low variance); a degree-10 model may overfit (low bias, high variance). As $n \rightarrow \infty$, variance shrinks and higher-degree models become viable.

Exercise 1.8.1

Prove the bias–variance decomposition (1.34) by expanding $\mathbb{E}_{\mathcal{D}}[(y - \hat{f}_{\mathcal{D}}(x))^2]$ and using the fact that ϵ is independent of $\mathcal{D}$.

Regularization

Regularization controls model complexity by adding a *penalty* $R(\theta)$ to the empirical risk:

$$C_\lambda(\theta) = \underbrace{\frac{1}{n} \sum_{i=1}^n \ell(y^{(i)}, f_\theta(x^{(i)}))}_{\text{empirical risk}} + \lambda R(\theta), \quad (1.35)$$

where $\lambda \geq 0$ is the *regularization hyperparameter*. Larger λ penalizes complexity more heavily, increasing bias but reducing variance. λ is not learned from the training data; it is selected via cross-validation.

Two penalties are standard.

Ridge regression (L_2 regularization).

$$R(\theta) = \|\theta\|^2 = \sum_{j=0}^p \theta_j^2. \quad (1.36)$$

Applied to linear regression with squared loss, the regularized objective is

$$C_\lambda(\theta) = \frac{1}{n} \|y - X\theta\|^2 + \lambda \|\theta\|^2,$$

which admits the closed-form solution

$$\hat{\theta}_\lambda = (X^\top X + n\lambda I)^{-1} X^\top y. \quad (1.37)$$

The penalty shrinks all parameters toward zero. Adding $n\lambda I$ makes $X^\top X + n\lambda I$ always invertible, improving numerical stability even when $X^\top X$ is singular.

Lasso (L_1 regularization).

$$R(\theta) = \|\theta\|_1 = \sum_{j=0}^p |\theta_j|. \quad (1.38)$$

The L_1 penalty promotes *sparsity*: many parameters are driven to exactly zero, effectively selecting a subset of features. No closed form exists; the problem is convex and solved via coordinate descent or proximal gradient methods.

Remark 1.8.2

In practice, the intercept θ_0 is typically excluded from the penalty, as shrinking it has no regularizing benefit. A combination of both penalties, $R(\theta) = \alpha \|\theta\|_1 + (1 - \alpha) \|\theta\|^2$ for $\alpha \in [0, 1]$, is known as the *elastic net* and inherits both sparsity (L_1) and stability (L_2) properties.

Remark 1.8.3

Regularization applies to all three models. For logistic and softmax regression, adding $\lambda\|\theta\|^2$ (L_2) or $\lambda\|\theta\|_1$ (L_1) to the cross-entropy loss yields a strictly convex objective (L_2) or a sparse solution (L_1), with no change to the gradient descent algorithm — only the gradient gains an extra term $2\lambda\theta$ or $\lambda\text{sign}(\theta)$ respectively.

Exercise 1.8.2

1. Derive the closed-form solution (1.37) by differentiating $C_\lambda(\theta)$ and setting the gradient to zero. Show that $X^\top X + n\lambda I$ is always invertible for $\lambda > 0$.
2. Explain geometrically why L_1 regularization promotes sparsity while L_2 does not. (*Hint*: consider the level sets of the two penalties and how they interact with the contours of the empirical risk.)

1.9 Gradient Descent

Linear regression with squared loss admits the closed-form solution (1.15), but logistic and softmax regression do not. More generally, closed-form solutions are unavailable when the loss is non-quadratic (e.g., L_1 , Huber, cross-entropy), when p is very large, or when data arrives in a stream. In all these cases, we minimize $C(\theta)$ iteratively using *gradient descent*.

The Algorithm

The core idea is simple: at each iteration, move θ in the direction of steepest descent of C .

Algorithm 1: Gradient descent

Input: Training data $\mathcal{D}$, objective $C(\theta)$, initial parameters $\theta^{(0)}$, learning rate $\alpha > 0$

Output: Approximate minimizer $\hat{\theta}$

```

1  $\theta \leftarrow \theta^{(0)}$ 
2 repeat
3   | Compute  $\nabla_\theta C(\theta)$ 
4   |  $\theta \leftarrow \theta - \alpha \nabla_\theta C(\theta)$ 
5 until termination condition is met
6 return  $\theta$ 
  
```

At iteration t , the parameter update is

$$\theta^{(t+1)} = \theta^{(t)} - \alpha \nabla_\theta C(\theta^{(t)}), \quad (1.39)$$

where $\alpha > 0$ is the *learning rate* (or *step size*), controlling how far we move along the gradient

direction. A simple and common stopping rule is

$$\|\theta^{(t+1)} - \theta^{(t)}\| < \varepsilon, \quad (1.40)$$

for a small tolerance $\varepsilon > 0$.

Remark 1.9.1

The learning rate α is critical: too small and convergence is slow; too large and the iterates may overshoot and diverge (see Figure 1.7). For deep models, $\nabla_{\theta} C(\theta)$ is computed efficiently via *backpropagation*, which applies the chain rule layer by layer.

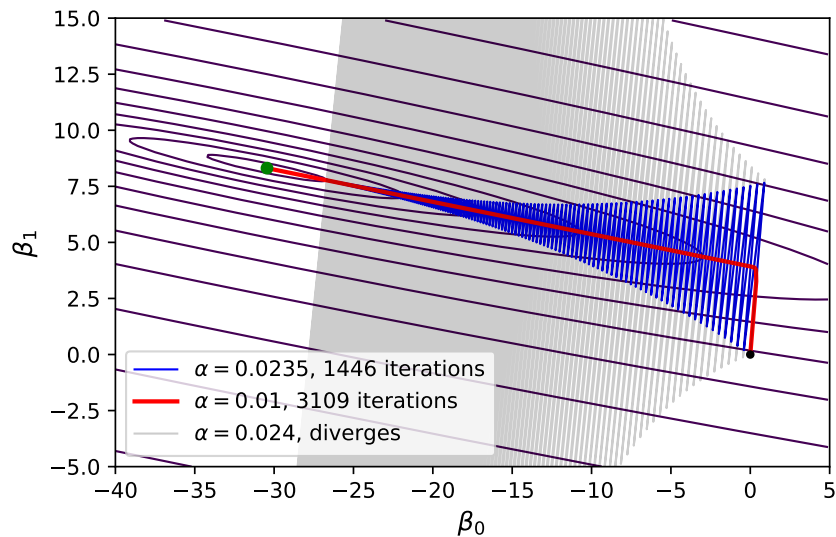


Figure 1.7: Effect of the learning rate on gradient descent for simple linear regression. Small α : slow convergence. Moderate α : fast convergence. Large α : divergence.

Gradients for Linear, Logistic, and SoftMax Regression

A key observation is that the gradients for all three models share a common structure. Let $X \in \mathbb{R}^{n \times d}$ be the design matrix with rows $\tilde{x}^{(i)\top}$, $y \in \mathbb{R}^n$ (or $Y \in \mathbb{R}^{n \times K}$) the label vector (or matrix), and $\hat{y}$ the vector of model predictions.

Linear regression. With $C(\theta) = \frac{1}{n} \|y - X\theta\|^2$ and $\hat{y} = X\theta$,

$$\nabla_{\theta} C(\theta) = \frac{2}{n} X^{\top} (\hat{y} - y). \quad (1.41)$$

Logistic regression. With $C(\theta)$ as in (1.21) and $\hat{y} = \sigma(X\theta) \in (0, 1)^n$ componentwise,

$$\nabla_{\theta} C(\theta) = \frac{1}{n} X^{\top} (\hat{y} - y). \quad (1.42)$$

The derivation uses the identity $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, which causes the sigmoid to cancel cleanly from the chain rule.

SoftMax regression. With $C(W)$ as in (1.28), $\hat{Y} \in \mathbb{R}^{n \times K}$ with rows $\hat{y}^{(i)\top} = \text{softmax}(W\tilde{x}^{(i)})^{\top}$, and $Y \in \mathbb{R}^{n \times K}$ with rows $y^{(i)\top}$ (one-hot),

$$\nabla_W C(W) = \frac{1}{n} (\hat{Y} - Y)^{\top} X, \quad (1.43)$$

which has the same dimensions as $W \in \mathbb{R}^{K \times d}$.

Remark 1.9.2

Unified structure. All three gradients take the form *(predictions minus labels)* times *inputs*, averaged over training examples. This is not a coincidence: it follows from the fact that all three losses are negative log-likelihoods paired with their natural output transformation (identity, sigmoid, softmax). This structure makes gradient descent straightforward to implement uniformly across all three models.

Exercise 1.9.1

1. Derive (1.42) by differentiating (1.21) with respect to θ , using $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.
2. Write out the gradient descent update (1.39) explicitly for logistic regression.
3. For SoftMax regression, write the update for the k -th row of W using (1.43).

Properties of Loss Functions

The convergence behaviour of gradient descent depends on the mathematical properties of the objective $C(\theta)$. Three properties play a central role in optimization theory.

Convexity. A differentiable function $C : \mathbb{R}^d \rightarrow \mathbb{R}$ is *convex* if for all $\theta, \theta' \in \mathbb{R}^d$,

$$C(\theta') \geq C(\theta) + \nabla C(\theta)^{\top} (\theta' - \theta). \quad (1.44)$$

The function lies above every tangent hyperplane, and every local minimum is a global minimum. C is μ -strongly convex (for $\mu > 0$) if

$$C(\theta') \geq C(\theta) + \nabla C(\theta)^{\top} (\theta' - \theta) + \frac{\mu}{2} \|\theta' - \theta\|^2, \quad (1.45)$$

which implies a unique global minimizer.

L -smoothness. C is L -smooth (for $L > 0$) if its gradient is L -Lipschitz:

$$\|\nabla C(\theta) - \nabla C(\theta')\| \leq L \|\theta - \theta'\| \quad \text{for all } \theta, \theta'. \quad (1.46)$$

Equivalently, C admits the quadratic upper bound

$$C(\theta') \leq C(\theta) + \nabla C(\theta)^\top (\theta' - \theta) + \frac{L}{2} \|\theta' - \theta\|^2. \quad (1.47)$$

This bound is the key tool for proving convergence: choosing step size $\alpha \leq 1/L$ guarantees each update decreases C .

Lipschitz continuity of the loss. The per-sample loss ℓ is G -Lipschitz if

$$|\ell(\theta) - \ell(\theta')| \leq G \|\theta - \theta'\| \quad \text{for all } \theta, \theta',$$

equivalently $\|\nabla \ell(\theta)\| \leq G$ almost everywhere. This bounds the magnitude of gradient noise in stochastic methods.

Example 1.9.1

For linear regression with squared loss, $C(\theta) = \frac{1}{n} \|y - X\theta\|^2$:

- C is L -smooth with $L = 2\lambda_{\max}/n$.
- When X has full column rank, C is μ -strongly convex with $\mu = 2\lambda_{\min}/n$.
- The *condition number* $\kappa = L/\mu = \lambda_{\max}/\lambda_{\min}$ measures the eccentricity of the loss landscape.

The binary cross-entropy and categorical cross-entropy losses are also convex (but not strongly convex) in θ when f_θ is linear.

Remark 1.9.3

Convergence rates. For gradient descent with step size $\alpha = 1/L$:

- *Convex, L -smooth:* $C(\theta^{(t)}) - C^* \leq \frac{L\|\theta^{(0)} - \theta^*\|^2}{2t}$ — an $O(1/t)$ rate.
- *μ -strongly convex, L -smooth:* $\|\theta^{(t)} - \theta^*\|^2 \leq \left(1 - \frac{\mu}{L}\right)^t \|\theta^{(0)} - \theta^*\|^2$ — a linear rate.

The linear rate $(1 - 1/\kappa)^t$ shows that a large condition number κ slows convergence considerably. These results will be derived rigorously in later weeks.

Exercise 1.9.2

1. Verify that the squared loss $C(\theta) = \frac{1}{n} \|y - X\theta\|^2$ is L -smooth with $L = 2\lambda_{\max}/n$ by bounding $\|\nabla C(\theta) - \nabla C(\theta')\|$ directly.
2. Show that C is μ -strongly convex with $\mu = 2\lambda_{\min}/n$ when X has full column rank. What happens to strong convexity when $n < d$?

Learning Rate Analysis for Linear Regression

For the linear model, the gradient descent dynamics can be analyzed exactly. Let $\lambda_{\max}$ be the largest eigenvalue of the Gram matrix $X^\top X$. Substituting (1.41) into (1.39) gives the affine iteration

$$\theta^{(t+1)} = \underbrace{\left(I - \frac{2\alpha}{n} X^\top X\right)}_A \theta^{(t)} + \frac{2\alpha}{n} X^\top y, \quad (1.48)$$

whose fixed point is any solution of the normal equations (1.14). The iteration converges to this fixed point if and only if the spectral radius $\rho(A) < 1$, which holds precisely when

$$\alpha < \frac{n}{\lambda_{\max}}. \quad (1.49)$$

Since the eigenvalues of A are $1 - 2\alpha\lambda_k/n$ for each eigenvalue λ_k of $X^\top X$, the spectral radius is $\rho(A) = \max_k |1 - 2\alpha\lambda_k/n|$, which is minimized by the choice $\alpha = n/(\lambda_{\max} + \lambda_{\min})$.

Exercise 1.9.3

1. Show that if $0 < \alpha < n/\lambda_{\max}$, then $\|\theta^{(t)} - \theta^*\| \leq \rho(A)^t \|\theta^{(0)} - \theta^*\|$ for any solution θ^* of (1.14).
2. Express $\rho(A)$ in terms of the eigenvalues of $X^\top X$ and α . What value of α minimizes $\rho(A)$, and what is the resulting convergence rate?

The Loss Landscape and Input Standardization

The convergence rate of gradient descent depends on the shape of the loss surface. For the quadratic loss, the level sets are ellipses whose eccentricity is determined by the ratio $\lambda_{\max}/\lambda_{\min}$ of the Gram matrix. Highly elongated ellipses (large ratio) force a small α and slow convergence.

Standardizing the input features, as described in Section 1.2, improves this ratio. For two standardized features with sample correlation $r \in [-1, 1]$, the Gram matrix is

$$Z^\top Z = n \begin{bmatrix} 1 & r \\ r & 1 \end{bmatrix},$$

with eigenvalues $n(1 \pm |r|)$ and ratio $R_Z = \lambda_{\max}/\lambda_{\min} = (1 + |r|)/(1 - |r|)$. When $|r|$ is small, $R_Z \approx 1$ and the level sets are nearly circular, leading to more stable and faster gradient steps (Figure 1.8).

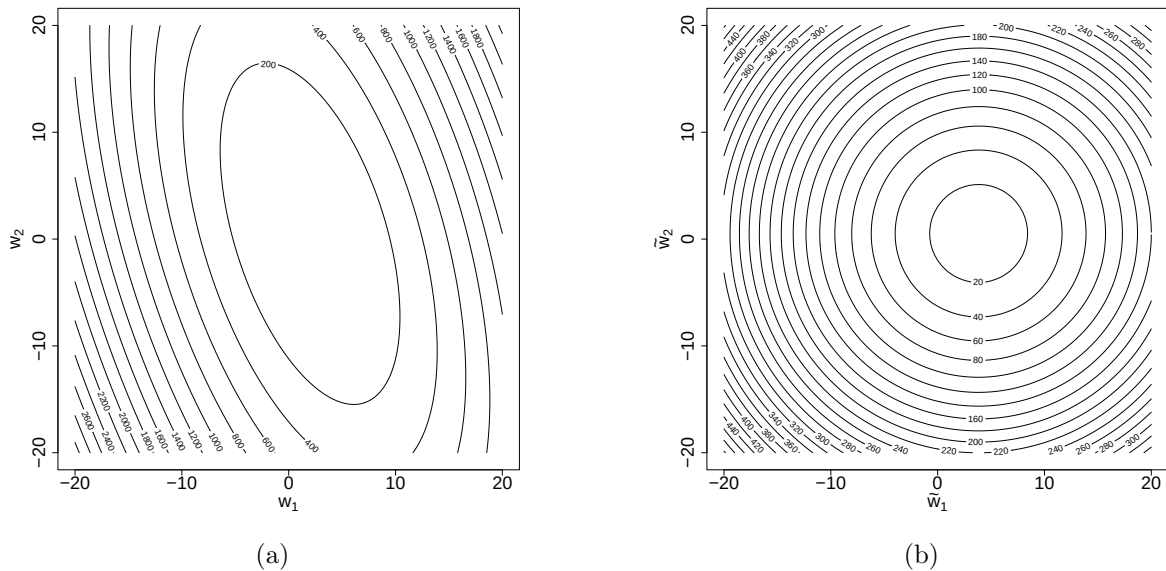


Figure 1.8: Loss contours for linear regression with $p = 2$. (a) Unstandardized inputs: elongated ellipses requiring a small step size. (b) Standardized inputs: rounder contours allowing larger, more stable steps.

Remark 1.9.4

When deploying a model trained on standardized data, apply the *same* transformation to new inputs using the mean $\bar{x}_i$ and standard deviation s_i computed from the training set — never from the test set.

Exercise 1.9.4

1. For linear regression with two standardized features (correlation r), find the optimal learning rate and the resulting convergence rate $\rho(A)$ in terms of r .
2. What happens to the convergence rate as $r \rightarrow 1$? Interpret this geometrically.

Stochastic Gradient Descent

Full gradient descent requires computing $\nabla C(\theta) = \frac{1}{n} \sum_{i=1}^n \nabla \ell(y^{(i)}, f_{\theta}(x^{(i)}))$, which costs $O(n)$ per iteration. For large datasets — modern deep learning routinely uses $n \sim 10^6$ to 10^9 — this is prohibitively expensive.

Stochastic gradient descent (SGD) replaces the full gradient with an inexpensive unbiased estimate computed on a randomly sampled *mini-batch* $\mathcal{B} \subset \{1, \dots, n\}$ of size $m \ll n$:

$$g^{(t)} = \frac{1}{m} \sum_{i \in \mathcal{B}^{(t)}} \nabla \ell(y^{(i)}, f_{\theta^{(t)}}(x^{(i)})). \quad (1.50)$$

Since $\mathcal{B}^{(t)}$ is sampled uniformly, $\mathbb{E}[g^{(t)}] = \nabla C(\theta^{(t)})$, so $g^{(t)}$ is an *unbiased estimator* of the full gradient. The parameter update is then $\theta^{(t+1)} = \theta^{(t)} - \alpha g^{(t)}$.

Algorithm 2: Mini-batch stochastic gradient descent (SGD)

Input: Training data $\mathcal{D}$, initial $\theta^{(0)}$, learning rate $\alpha > 0$, mini-batch size m

Output: Approximate minimizer $\hat{\theta}$

```

1 repeat
2   Sample mini-batch  $\mathcal{B} \subset \{1, \dots, n\}$  of size  $m$  uniformly at random
3    $g \leftarrow \frac{1}{m} \sum_{i \in \mathcal{B}} \nabla \ell(y^{(i)}, f_{\theta}(x^{(i)}))$ 
4    $\theta \leftarrow \theta - \alpha g$ 
5 until termination condition is met
6 return  $\theta$ 
  
```

The special case $m = 1$ is *pure SGD*; $m = n$ recovers full gradient descent. In practice m is a power of 2 (e.g., 32, 128, 256) to exploit hardware parallelism. One full pass through all n training points (i.e., $\lfloor n/m \rfloor$ mini-batch steps) is called an *epoch*.

Remark 1.9.5
SGD vs. full gradient descent.

- *Cost per step:* SGD costs $O(m)$ vs. $O(n)$, enabling roughly n/m more steps per unit time.
- *Convergence (convex):* With constant step size, SGD converges at rate $O(1/\sqrt{t})$, slower than the $O(1/t)$ of full GD. A decaying schedule $\alpha_t \rightarrow 0$ recovers $O(1/t)$.
- *Non-convex objectives:* Gradient noise can help SGD escape sharp local minima and saddle points — a key practical advantage over full GD for deep networks.
- *Practice:* SGD and its adaptive variants (e.g., Adam, RMSProp) are the workhorses of deep learning.

Exercise 1.9.5

1. Show that the mini-batch gradient (1.50) is an unbiased estimator of $\nabla C(\theta)$ when $\mathcal{B}$ is sampled uniformly at random from $\{1, \dots, n\}$.
2. For $n = 10,000$ and $m = 100$, how many mini-batch steps constitute one epoch? How does the total computational cost of one epoch of SGD compare to one step of full gradient descent?